

QUESTIONS

Q1

Animal Sound Debugging
20 marks

Q2

Static Object Counter
20 marks

Q3

Bank Account Runtime
Polymorphism
20 marks

Q4

Deep Copy of Student
Record
20 marks

Q5

Payment Processing System
20 marks

5/5 answered

Q1 Animal Sound Debugging

20 marks

Debug the Animal hierarchy so that Dog, Cat, and Cow override sound(). The correct derived sound must be produced through an Animal pointer.

Your Code

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4 class Animal {
5 public:
6     virtual void sound() {
7         cout << "Animal makes a sound" << endl;
8     }
9     virtual ~Animal() {}
10 };
11 class Dog : public Animal {
12 public:
13     void sound() override {
14         cout << "Bark" << endl;
15     }
16 };
17 class Cat : public Animal {
18 public:
19     void sound() override {
20         cout << "Meow" << endl;
21     }
22 };
23 class Cow : public Animal {
24 public:
25     void sound() override {
26         cout << "Moo" << endl;
27     }
28 };
29
30 int main() {
31     string input;
32     if(!(cin >> input)) {
33         Animal* a1 = new Dog();
34         Animal* a2 = new Cat();
35         Animal* a3 = new Cow();
36         a1->sound();
37         a2->sound();
38         a3->sound();
39         delete a1; delete a2; delete a3;
40     } else {
41         Animal* a = nullptr;
```

QUESTIONS

Q1

Animal Sound Debugging
20 marks

Q2

Static Object Counter
20 marks

Q3

Bank Account Runtime
Polymorphism
20 marks

Q4

Deep Copy of Student
Record
20 marks

Q5

Payment Processing System
20 marks

5/5 answered

Q2 Static Object Counter

20 marks

Create a static data member in Employee to count all Employee objects created. The count must be shared across objects and accessible without maintaining a separate counter in each object.

Your Code

```
1 #include <iostream>
2 using namespace std;
3 class Employee {
4 private:
5     static int count;
6 public:
7     Employee() {
8         count++;
9     }
10    static int getCount() {
11        return count;
12    }
13    void displayCount() {
14        cout << count << endl;
15    }
16 };
17 int Employee::count = 0;
18 int main() {
19     int n;
20     if(cin >> n) {
21         for(int i=0; i<n; i++) {
22             Employee e;
23             if(i == n-1) e.displayCount();
24         }
25     } else {
26         Employee e1, e2, e3;
27         e3.displayCount();
28     }
29     return 0;
30 }
```

Input

stdin input

Output

QUESTIONS

Q1

Animal Sound Debugging
20 marks

Q2

Static Object Counter
20 marks

Q3

Bank Account Runtime
Polymorphism
20 marks

Q4

Deep Copy of Student
Record
20 marks

Q5

Payment Processing System
20 marks

5/5 answered

Q3 Bank Account Runtime Polymorphism 20 marks

Write a C++ program using an abstract Account class and derived SavingsAccount, CurrentAccount, and LoanAccount classes. Implement calculateInterest() using runtime polymorphism. Identify and correct the starter-code error so a base pointer invokes the derived implementation.

Your Code

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4 class Account {
5 protected:
6     double balance;
7 public:
8     Account(double b) : balance(b) {}
9     virtual double calculateInterest() = 0;
10    virtual ~Account() {}
11 };
12 class SavingsAccount : public Account {
13 public:
14     SavingsAccount(double b) : Account(b) {
15         double calculateInterest() override {
16             return balance * 0.04; // 4%
17         }
18 };
19 class CurrentAccount : public Account {
20 public:
21     CurrentAccount(double b) : Account(b) {
22         double calculateInterest() override {
23             return balance * 0.02; // 2%
24         }
25 };
26 class LoanAccount : public Account {
27 public:
28     LoanAccount(double b) : Account(b) {}
29     double calculateInterest() override {
30         return balance * 0.10; // 10%
31     }
32 };
33 int main() {
34     string type;
35     double bal;
36     if(!(cin >> type)) {
37         Account* a1 = new SavingsAccount(10
```

QUESTIONS

Q1

Animal Sound Debugging
20 marks

Q2

Static Object Counter
20 marks

Q3

Bank Account Runtime
Polymorphism
20 marks

Q4

Deep Copy of Student
Record
20 marks

Q5

Payment Processing System
20 marks

5/5 answered

Q4 Deep Copy of Student Record

20 marks

Debug a class containing dynamically allocated marks. Implement a deep-copy constructor and destructor. Changing one object's marks must not change the copied object.

```
class Student {  
    int* marks;  
public:  
    Student(int m) { marks = new int(m); }  
    Student(const Student& s) {  
        marks = s.marks; // Bug: shallow copy  
    }  
    void setMarks(int m) { *marks = m; }  
    void display() { cout << *marks << endl; }  
};
```

Your Code

```
1 #include <iostream>  
2 using namespace std;  
3 class Student {  
4     int* marks;  
5 public:  
6     Student(int m) {  
7         marks = new int(m);  
8     }  
9     Student(const Student& s) {  
10         marks = new int(*s.marks);  
11     }  
12     ~Student() {  
13         delete marks;  
14     }  
15     void setMarks(int m) {  
16         *marks = m;  
17     }  
18     void display() {  
19         cout << *marks << endl;  
20     }  
21     int getMarks() { return *marks; }  
22 };  
23 int main() {  
24     int m1, m2;  
25     if(!(cin >> m1)) return 0;  
26     Student s1(m1);  
27     Student s2 = s1;
```

QUESTIONS

Q1

Animal Sound Debugging
20 marks

Q2

Static Object Counter
20 marks

Q3

Bank Account Runtime
Polymorphism
20 marks

Q4

Deep Copy of Student
Record
20 marks

Q5

Payment Processing System
20 marks

5/5 answered

Q5 Payment Processing System

20 marks

Design an abstract Payment class with a processPayment(). Implement CreditCard, UPI, NetBanking, and Wallet. Use runtime polymorphism and avoid a switch-case statement based on payment type.

Your Code

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4 class Payment {
5 public:
6     virtual void processPayment() = 0;
7     virtual ~Payment() {}
8 };
9 class CreditCard : public Payment {
10     double amount;
11 public:
12     CreditCard(double a = 0) : amount(a) {}
13     void processPayment() override {
14         if(amount > 0)
15             cout << "Processing Credit Card
16         else
17             cout << "Processing Credit Card
18     }
19 };
20
21 class UPI : public Payment {
22     double amount;
23 public:
24     UPI(double a = 0) : amount(a) {}
25     void processPayment() override {
26         if(amount > 0)
27             cout << "Processing UPI Payment
28         else
29             cout << "Processing UPI Payment
30     }
31 };
32
33 class NetBanking : public Payment {
34     double amount;
35 public:
36     NetBanking(double a = 0) : amount(a) {}
37     void processPayment() override {
38         if(amount > 0)
39             cout << "Processing NetBanking
```